

Data Warehousing and Mining

Relational DB: Consistency and integrity crucial, simple queries, data from single area

Data Warehouse: Complex queries, collection of data that supports management decisions. Subject orientated, integrated, time variant, non-volatile. repository of information collected from multiple sources, stored under a unified schema. non-trivial, implicit, previously unknown and potentially useful

OLTP: Online transaction processing. Major task of RDBMS, day to day operations, many short transactions, queries touch small amounts of data, updates are frequent, concurrency is performance concern.

OLAP: Online analytical processing. Major task of data warehouse, data analysis and decision making, organise and present different ways, long transactions, complex queries, lots of data touched, infrequent updates, Multi-dimension, requires lots of resources

DW Applications

1. **Information Processing:** Basic stats, cross tabs, charts
2. **Analytical processing:** Multi-dimensional analysis, OLAP (slice-dice, drilling, pivoting)
3. **Data mining:** Knowledge discovery from hidden patterns, construct model, prediction, classify ,support associations, knowledge mining

GROUP BY ROLLUP(A,B,C)

- A,B,C

- (A,B) subtotals

- (A) subtotals

- Total

GROUP BY CUBE(A,B,C)

- A,B,C

- Subtotals for the following:
(A,B), (A,C), (B,C),
(A), (B), (C)

- Total

Practical Concerns

Date/Time: allows trend analysis and time series, typically 1 row a day. **Properties:** indicators for certain periods. **Pos:** Use date and time dimensions over SQLtimestamp as captures more detailed properties, avoids SQL functions, make use of indexes. Keep Datetime separate

Surrogate key: PK of dim table is surrogate, not natural key.

Surrogate key: meaningless integer assigned by DW. **Natural Key:** Meaningful to users (e.g. SKU, account). Synthetic vs Semantic. **Benefits Surrogate:** insulates DW from changes to operational systems, easier to integrate multiple sources, reduced storage space->better performance, better handle exception cases NaN. **Degenerate Dimension:** Dimension key without corresponding table, just identifier without attr.

Dim Performance: OLAP I/O bound, data intensive, disk is bottleneck, keep data size small, correlate attributes together.

Slowly Changing Dimensions: Dimensions don't change as much as fact. Methods: 1. Overwrite history: simple to implements 2. Add new dimension row: accurate historical reporting, pre computed aggregates unaffected, dimension table grows over time, requires surrogate key 3. Able to report using either old or new values, create two columns, previous district and current district, good for corporate reorganisations.

Role play Dimension: When 1 thing is used twice in different DimTables, make new DimTable that has its foreign key in original DimTables

Junk Dimension: some attributes don't fit nicely, grouping of low-cardinality flags and indicators, works best if leftover attributes are few in number, low cardinality, correlated.

Measures in Fact: Additive: easy to work with, summing values give meaningful result(quantity, total sales). Non-additive: averages, percentages, BIG DISADV- cannot pre-compute aggregates. Store additive: store numerators and denominator, can used for distributive aggregates

Transaction Facts: Each row is a discrete event, most granular, detailed information

Snapshot: each row is a point-in-time snapshot, taken at predefined time intervals, provides cumulative view, used for continuous processes/measures of intensity, facts are semi additive-> measures are additive across some dimensions but not all(add across non-date dim, not add across date dim)

Factless Fact Table: capture relationships between dimensions with no numeric column, still needed for analysis, include dummy column value=1, Transactional fact tables don't have rows for non-events. Good: Take advantage of sparsity. Bad: No record of non-events. See non-events by comparing to all possible events.

Multiple Currencies: global businesses have different currencies, different time zones, reporting requirements may be diverse(Standard currency vs. local currency, depends on stakeholder, can standardise times(GMT) but buying patterns can be different. Add Currency dimension to fact table, for local and standard, create currency conversion table(rate on the day transaction happened)

Data Warehousing Architecture

1. Data Sources

Discovery: Online extraction(direct connection to source data), Offline extraction(out side original source system), Full extraction (completely sourced from source system, no need for tracking), Incremental extraction(different sources for same data, need to track different sources)

Extract: Fetching relevant data from source system put into data staging environment, anomaly detection, logical data map (start and end points of columns for all relevant data, meta data, data profile)

2. Data Staging

Don't have extra activities to convert and transform data during initial extraction to speed up the process, don't load data into memory all at once strains server resources

Transform: Applying transformations to the data to make it high quality and fits original purpose. Consistent, not incomplete, not wrong, there are no duplicates or out of data data. Column property enforcement, enforcing cleansing rules, logical data checks. Use pandas and python, join tables with surrogate keys

3. Data Warehouse

Create dimension and fact tables with surrogate keys that auto increment, insert data into DW.

Load: Load data into warehouse(fact and dim), build additional structures to improve system performance, consistent and reliable format

Validation

Using rules to prevent poor quality data from entering DW: Incoming validation (without comparing to DW, ranges, formats, valid), Cross reference(compare incoming data with data with DW data, during loading process), Internal validation (check loaded properly)

More useful cypher

NOT, HAS, STARTS WITH, ENDS WITH, CONTAINS

ROLLUP vs CUBE: cube computes entire lattice, rollup computes one path through lattice(group by order matters)

Modelling DW OLAP

Storing data in DW: Advanced management system on storing large amounts of data.

- Built on top of RDBMS (SQL)
- Store using tables (multidimensional)
- Data cube used to support OLAP
- Organised around central theme

Data cube: data storage used for modelling and viewing in multidimension (5D is series of 4D)

Fact Tables: contain measures of a central theme and keys to dimension tables: narrow, many rows, numeric, growing over time, obtain or aggregate measurements.

Measurements: numeric function that used to evaluate each point in data cube space. Aggregate data corresponding to respective dim-value pairs

- **Distributed:** count, min, max, sum
- **Algebraic:** computed with M arguments form distributed measures: avg, std
- **Holistic:** median, mode, rank

Dimension Tables: correspond to real world object/concept, many descriptive columns, not many rows. Uses: Filters based on dimension table attributes, grouping based on dimension, fact tables reference dim. Determine candidate key based on grain, add relevant dimensions based on candidate key.

Role of Concept Hierarchies

Concept Hierarchies: Defines sequence of mappings from low to high level

Grouping Hierarchies: Grouping values for a given dimension or attribute

Schema of DW

Star schema: fact table in middle connected to dim tables -> fewer tables = faster, more redundant information

Snowflake Schema: Some dimensional hierarchy is normalised into a set of smaller dim tables -> reduces redundancy, less efficient = more tables

Galaxy Schema: multiple fact tables share dim tables

OLAP Operations -> Cross tabulation

Rollup: further aggregation, summarise data, climbing up hierarchy or dimension reductions

Drill Down: less aggregation, reverse of roll up, higher to lower level, more detailed -> new dims

Slice and Dice: project and select (regular SQL)

Slice=Pick rectangular subset of cube (then dice)

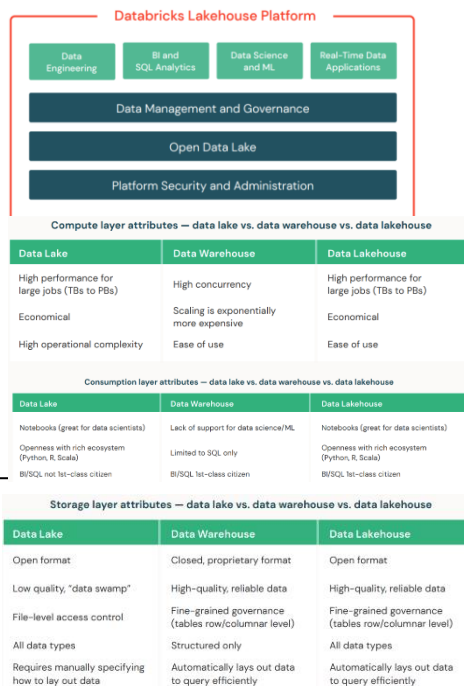
Dice=Sub cube of specific values of multiple dims

Pivot: reorientate the cube, switching axis

Drill Across: multiple fact tables->merge 2 queries

Data Lake: storage platform, flexible data, specific purpose of supporting execution of analytic workloads, stored in raw format. Uses: exploratory data science, big data processing, machine learning, real-time. Compliment to DW

Data Lakehouse: combines DW and DL, offering analytics flexibility with diverse data types



APOC

More powerful tools for importing/exporting data, Text parsing functions, Path expansion, Virtual graphs

Virtual Nodes and Relationships

APOC allows us to create virtual nodes and relationships, which don't exist in the graph — they are stored in memory and returned to the UI, representing transitive relationships between nodes

MATCH (from:Account)-[:SENT]->(p:Payment)-[:RECEIVED]->(to:Account)

RETURN from, to, apoc.create.vRelationship(from:'PAID',{amount:sum(p.amount)),to) as rel;

Multi-dimensional data model

Hierarchies

StarNet Model: radial lines emanating from centre, each line is a concept hierarchy for a dimension, each level is a footprint, represent granularities for OLAP operations (finest grain question we can ask?)

DW Design template – Kimball's four steps:

1. **Identify business process to model:** sales
 2. **Determine the grain** at which facts will be stored
 - o Finer grain expensive
 3. **Choose dimensions** that apply to fact rows
 - o Determine candidate key based on grain
 4. **Identify measure** that populates fact table rows
- Must balance business requirements and data realities (what you can put in it)
 - Trade-off between performance and grain
 - o Better to use finer grain if in doubt
 - o Precomputing aggregates can solve performance issues -> summary table

Data cube

Cells: unique logical intersection in a cube of one member from every dimension in the cube

What is a data cube: Organised around central theme, allows data to be modelled and viewed in multiple dimensions, constructed from F & D tables, **lattice of cuboids**

Cuboids in a lattice: each cuboid corresponds to a different degree of summarisation of multi-dim data, 2nd number of cuboids, Apex 0-D cuboid is "all" and is highest summary, Base cuboid is specific and least summary

Curse of dimensionality: add more dimensions increases detail but decreases performance by 2^D

Meta Data: Data about data. Store descriptive information about data model to store and share. Helps organise, find, understand data and its detail. Acts as road-map and directory for DW, defines DW objects. Descriptions, categories, time, location, last access, who can access. Describes tables and columns

Kinds of metadata: **data definitions and mapping** (meaning of each column and location), **data structure** (schema), **source system** (initial structure in source system), **ETL** (data flow in ETL process), **data quality** (rules, risk levels, and their actions), **audit data** (record of processes and activities in the DW), **Usage data** (logs of app use)

Definition data: list of all columns from DDS, ODS, and NDS. Uses table and column key defined in data structure metadata

table	key	column	key	description	sample	value	source	column	key
56	112			The surrogate key of the product dimension. It is unique, is not null, and is the primary key of the product dimension.	3746	88			

Mapping data: lineage map, from source to DW

data	mapping	key	column	key	source	column	key	create	timestamp	update	timestamp
1	378	249			2007-10-24 09:23:48	2007-11-18 14:10:08					

Metadata repository: Metadata collectively organised in a catalogue. Stores info about model. One or more each DW. **Stores:** definition, structure, description, outcomes(indexing, ETL), access, aggregation/summarising

Categories: business, operation, technical

1) Business Metadata includes data ownership information, business definition, and changing policies.

- ❑ Description of information from the business perspective (e.g., weekly sales, or budget variance reports)
- ❑ Contains high-level definitions of all fields present in the data warehouse, information about cubes, aggregates, Data Marts.
- ❑ addressed to and used by the data warehouse users
- ❑ report querying authors, cubes creators, data managers, testers, analysts

2) Technical Metadata includes structural information such as primary and foreign key attributes and indices, data types and values.

- ❑ gives information about the **structure** of data, where it **resides**, and other **technical** details related to finding data in its native database.
- ❑ mainly used by **software tools** to understand and process data
- ❑ stores data **mapping and transformations** from source systems to the data warehouse
- ❑ illustrates information related to **system functions and metadata**

3) Operational Metadata offers a connection between the metadata repository and the data warehouse

- ❑ Allows adding physical database columns to the data warehouse tables
- ❑ includes currency of data and data lineage and enables easier use for **business and technical** consumers.
- Currency of data means whether the data is active, archived, or purged
- Lineage of data means the history of data migrated and transformation applied on it
- ❑ **Benefits** of operational metadata
 - ❑ referenced at a row level of granularity in DW (unlike in meta data repository)
 - ❑ provides a detailed row level explanation of actual information content

Roles of Metadata

Managing: Best managed with: Individual side requires that people be trained in the importance and use of metadata, Process side incorporates metadata management into the data warehousing and business intelligence life cycle, Tools (manager, repository, metadata extract tools, modelling and ETL tools, BI reporting tools)

Challenges: Non-homogenous information sources(hard to maintain consistent format), enforcing business rules(clarifying dependencies in data and understanding relationships is hard-> need to provide context), good communication (no industry standards)

Frequent Itemset and Associate Rule Mining Pattern Analysis: frequent pattern, set of items, subsequences, substructures that occurs frequently together (or strong correlation) in data set. E.g. What is purchased together?. Important: association, correlation, causality, basket data analysis->Important intrinsic property from dataset

Itemset: A set, Tid, of 1 or more items, length is K
Absolute Support: frequency or number of occurrences in itemset X. The count.

Relative Support: Fraction of transactions that contain X (probability). $3/5 = 60\%$

Min Support: Itemset is frequent if X's support is no less than a minsup threshold. Denotes if frequent or not.

Patterns: Many frequent patterns exist, a long pattern contains frequent sub patterns

Closed patterns: A pattern (itemset) X is closed if X is frequent and there exists no super-pattern $Y \supset X$, with the same support as X. Closed pattern is a lossless compression of frequent patterns

->Reduces the # of patterns but does not lose the support information!

Max patterns: A pattern (itemset) X is a max-pattern if X is frequent and there exists no frequent super-pattern $Y \supset X$. Max-pattern is a lossy compression->we only know max is frequent, don't know real support of max anymore
Max VS Closed: closed is more desirable than max, in max you don't know the frequency of any subset itemset. Max is closed but not all closed are max.

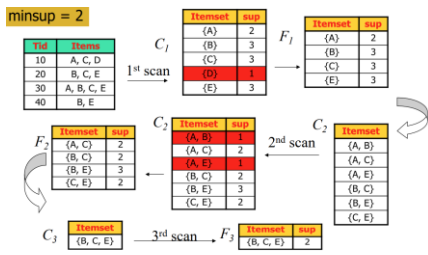
Apriori Algorithm: monotonicity, Any subset of a frequent itemset must also be frequent: **down closure property**. Efficient mining method (Apriori pruning principal): Any super set of an infrequent itemset must also be infrequent -> If any subset of an itemset S is infrequent then there is no chance for S to be frequent - don't consider S (sharp knife for pruning)

Aprior Implementation: level wise, candidate generation, testing.

C_k : candidate k-itemsets
 F_k : frequent k-itemsets

$k = 1$;
 $F_1 = \{\text{frequent items}\};$ //frequent 1-itemset

```
for (k = 2;  $F_{k-1} \neq \emptyset$ ; k++) do {  
    /* candidates generation */  
     $C_k = \{\text{candidates generated from } F_{k-1}\};$   
  
    /*  $F_{k+1}$  = candidates in  $C_{k+1}$  with minsup */  
    Derive  $F_k$  by counting candidates in  $C_k$  w.r.t. DB at minsup;  
}  
return  $\cup_k F_k$ ;
```



Associate Mining Rules:

Support: prob a transaction contains $\{X\} \cup \{Y\}$
$$\text{support}(X \Rightarrow Y) = \frac{P(X \cup Y)}{P(X \cup Y)} = \frac{\text{Trans with X and Y}}{\text{Total Trans}}$$

Confidence: conditional probability that transaction with X contains Y.

$$\text{Confidence}(X \Rightarrow Y) = \frac{P(X|Y)}{P(X|Y)} = \frac{\text{Trans with X and Y}}{\text{Trans with X}}$$

Lift: Asses the degree to which occurrence of one "lifts" the other, centred around 1.

$$\text{lift}(X, Y) = \frac{P(X \cup Y)}{P(X)P(Y)} = \frac{\text{Trans with X and Y}}{\text{Trans with X} \times \text{Trans with Y}}$$

Association rules: For each frequent itemset, F, generate all non-empty subsets of F, for each non-empty subset, s of F, output the rule:

$$s \Rightarrow (F - s) \text{ if } \frac{\text{support_count}(F)}{\text{support_count}(s)} \geq \text{min_conf}$$

Challenges: multiple scans of transactions, huge number of candidates, tedious work load of support counting for candidates

Improving Apriori: reduce passes of transaction database scans, shrink number of candidates, facilitate support counting of candidates

Implement improvements: Reduce passes(partitioning, dynamic itemset counting), shrink number of candidates(hash map, sampling, transaction reduction(has to contain previous frequent itemset

Data Mart: subset of existing DW orientated to specific business line, created by top down approach, built with star schema for data cube, improves response times due to reduction of data, easy access to frequently requested data, easy to implement, agile, data is partitioned and allows granular access control privileges. **DW enterprise wide, DM single subject**

When to consider data mart:

- If you want to partition the data with a set of **user access control** strategy.

- If a particular department wants to see the **query results much faster** instead of scanning huge Data Warehouse data.

- If a department wants data to be **built on other hardware (or) software** platforms.

- If a department wants data to be designed in a manner that is **suitable for its tools**.

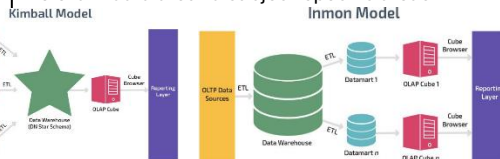
Types of DM: dependent, independent, hybrid
Dependent: source from single DW, benefits of centralisation, custom user privileges (user can access DW and DM or user only access DM). Pro: considered better because DW linked, no historical limit, improve data analysis quality, data integrity is ensured. Con: take longer, more expensive to implement

Independent: created without central DW, ideal for small groups in organisation, no relationship with DW, autonomous from business, opposes DW motivations->need consistent central store of enterprise data for multiple users. Pro: easier and faster implement, more flexible, Con: easy to form an information island, not true enterprise wide solution can become costly over time, don't provide historical depth of DW.

Hybrid: combines data from sources and DW, useful for ad-hoc integration for new products or group ect, best suited for multiple database environments and fast implementation(requires least data cleansing, supports large storage structures, flexible smaller applications)

Immon Model: top down approach, DW physical representation of the model

Kimball Model: Creates DMs that aggregate relevant data around subject-specific areas.



Stages: designing, constructing, populating, accessing, managing

Adv: managing big data and gaining valuable insights, efficient access, inexpensive, makes departments responsible for their maintenance

DisAdv: Creating too many becomes cumbersome -> disparate and unrelated DM, can't provide companywide analysis, for small business needs(up size->down performance)

Create or Merge

1. LOAD CSV WITH HEADERS FROM 'file:///Club.csv' AS row
MERGE (p:Club (name: row.name))
2. LOAD CSV WITH HEADERS FROM 'file:///Player_Number_r1.csv' AS row
MATCH (player: Player {playerid: toInteger(row.playerid)})
MERGE (player) -[:JERSEY_NUM_IS]-> (number)
3. CREATE (p:Person (name: "Siri"))

Read

1. MATCH (p1:Person)-[:1:ACTED_IN]-[:m:Movie]<-[:2:ACTED_IN]-[:p2:Person]
WITH p1, p2, toUpper(p1.name) AS upperCaseName
WHERE upperCaseName STARTS WITH "MARTIN"
RETURN p1, p2

Update

1. MATCH (p:Person (name: "Maetin Sheen"))
SET p.birthdate="date('1980-01-01')"

Delete

1. MATCH (p:Person (name: "Martin Sheen"))
REMOVE p.age
2. MATCH (p:Person (name: "Martin Sheen"))
DETACH DELETE p

Path Expansion

- (a)-[:5]->(b) Paths of length 5
- (a)-[:*5]->(b) Paths of length ranging from 0 to 5 inclusive
- (a)-[:*5..]->(b) Paths of length 5 or more
- (a)-[:*3..5]->(b) Paths from length 3 to 5 inclusive

Other

COUNT(), COLLECT(), AVG(), MAX(), SUM(), stDev(), DISTINCT LABELS/TYPE(n/r)

List Comprehension -> creates list based on pattern or existing list

1. MATCH (Keanu:Person (name: "Keanu Reeves"))
WITH Keanu, [(Keanu->[:b:Movie]) b.title] b.title AS movieTitles
SET Keanu.resume = movieTitles

2. [x IN Keanu.resume WHERE x contains "The Matrix"] as matrixList

APOC Awesome Procedures -> powerful data, text parsing, path, virtual graphs

Procedures: return a list that are unwound Replace need to write functions

Functions: simpler, typically return a single value

APOC CSV Importing nodes and relationships

CALL apoc.import.csv()

[[{fileName: "file:/persons.csv", labels: ["Person"]}],
[[{fileName: "file:/knows.csv", type: "KNOWS"}],
{delimiter: ";", arrayDelimiter: ";", stringIds: false}]

Text Functions

apoc.text.fuzzyMatch(s1,s2), apoc.text.clean(text), apoc.text.fuzzyMatch(string1, string2), apoc.text.indexOf(text, searchString)

Path Expansion APOC

CALL apoc.path.expand(startNode <id> [Node]list, 'TYPE[TYPE_OUT]-[:TYPE_IN', '+YesLabel|-NoLabel', minLevel, maxLevel) yield path

MATCH (p:Person (name: "Alicia"))

CALL apoc.path.subgraphAll(a, {relationshipFilter: 'FRIENDS_WITH', labelFilter: 'Person', minLevel: 0, maxLevel: 3

}) YIELD nodes, relationships

RETURN nodes, relationships

What is a Graph?: collection of vertices and edges know as nodes(nouns) and relationships (verbs). Can model scenarios using graphs - social networks, natural language, genes

Labels: nodes can have multiple labels, use labels to make node groups and query for label
Relationships: naturally form paths, makes graph DB extremely efficient for querying(more so when data is heavily connected)

Property graph model: common, contains nodes and relationships, node may have 0 or more properties, relationships can be names and directed and always have start/end node, relationships also contain properties

Relational VS Graph:

Graph DBs excel at dealing with highly connected data RDB can't handle relationships well, GDBs do not need to store null values, GDBs are agile meaning that new properties new node and relation types can be added on the fly without designing a schema apriori, GDBs are better at handling recursive queries and queries across multiple entities, GDBs are better at recursive querying and across multiple entities, RDB schemas are inflexible GDS has no schema

Graph Modelling: unruly domain to structure that can be manipulated. 1. On white board(using arrows tool), plan nodes, relationships, properties, labels. 2. Directly from relational tables: table to entity types, rows to nodes, column names to properties, cells to property values, a join or a foreign key is a relationship 3. Decide on whether the information should be modelled as a node or a property value.

Search: Process of systematically exploring a problem space to find a solution or goal.

Search Tree: A "what if" tree of plans and their outcomes, the start/initial state is the root node, children correspond to successors, nodes show states. **Process:** 1. Formulate problems into states, actions, transition model and action cost function 2. Use search algorithm to find solution **Pseudocode:** Same for all algorithms. 1. Frontier: nodes that are considered and candidates for further exploration in search of goal state. 2. Tie breaking by alphabet 3. Goal check after dequeue, not during enqueue

```
// start is the initial or start state, goal is the goal state  
function search_algorithm(start, goal):  
    frontier = Queue() //BFS, DFS, A* have different dequeue techniques  
    frontier.enqueue(start)  
    while not frontier.isEmpty():  
        current = frontier.dequeue()  
        if current == goal:  
            return "Goal found"  
        for neighbor in current.neighbors():  
            frontier.enqueue(neighbor)  
    return "Goal not found"
```

Breadth First Search BFS

Starts from chosen node and explores nodes 1 hop away before exploring 2 hops away, FIFO, basis for other path finding algorithms

Depth First Search DFS

LIFO, goes down children then tie breaks (priority and frontier)->then goes to winner child and tie breaks...., if not down that path it goes back up

A* Search

Allows inclusion of extra information that the algorithm can use with a heuristic function to determine which nodes to explore next. A* selects path that minimises $f(n) = g(n) + h(n)$, where $g(n)$ is the cost of the path from the source node, n , so far ($g(n)$ is the path that is additive) and $h(n)$ is the estimated cost of the path to the destination node ($h(n)$ changes depending on node not additive), $f(n)$ is estimated total cost of path through n to goal, lower $f(n)$ is better.

Minimum Spanning Tree MST

useful for waterpipes, starts from given node and finds all its reachable nodes and the set of relationships that connect the nodes together with minimum possible weight. Prim's Algo: 1. Start with chosen node 2. Smallest weight connected to the node is added to tree along with its connected node 3. Repeat process always choosing the minimal weight relationship that joins any node not already in the tree 4. When there are no more nodes the tree is in minimum spanning tree

Random Walk

Provides set of nodes on a random path in a graph. Know where they want to go but no strategy on how to get there. Useful for machine learning(node2vec, graph2vec algo, similarity)

Graph Data Science

